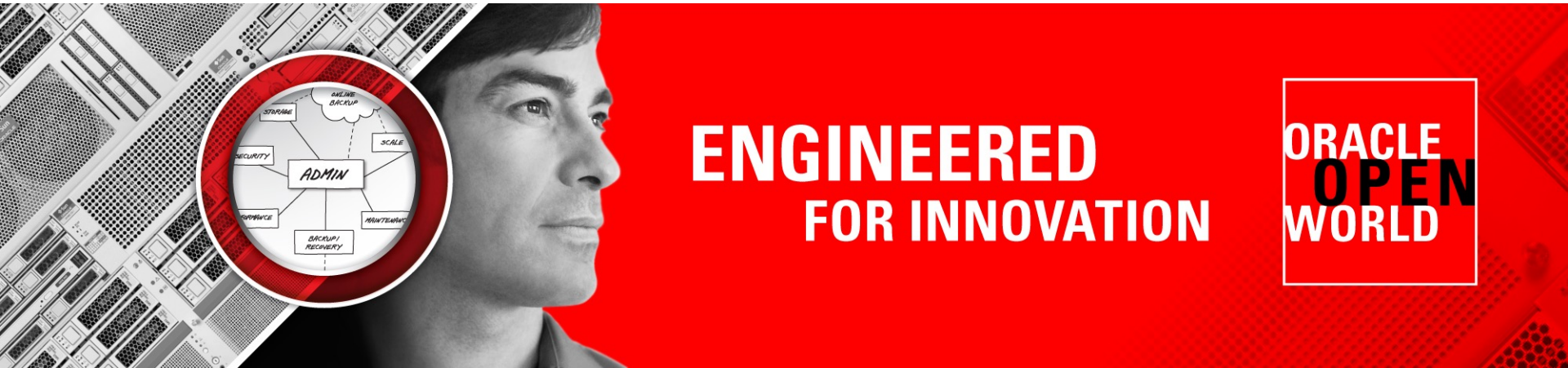




**ENGINEERED
FOR INNOVATION**

**ORACLE
OPEN
WORLD**

Cours PL/SQL

Présenté par: Mme Olfa DRIDI



Les jointures

Jointure

- Il est nécessaire de faire une jointure si l'on a besoin d'informations corrélées disséminées dans plusieurs tables :

```
SELECT Employe.numero, Departement.nomdep  
FROM Employe, Departement  
WHERE Employe.depart = Departement.numdep ;
```

- L'utilisation de l'alias permet une simplification d'écriture des requêtes.

```
SELECT E.numero, E.nom, D.numdep, D.nomdep, D.lieu  
FROM Employe E, Departement D  
WHERE E.depart = D.numdep ;
```

CROSS JOIN (produit cartésien)

- La commande CROSS JOIN est un type de jointure sur 2 tables SQL qui permet de retourner le produit cartésien.

```
SELECT * FROM table1 CROSS JOIN table2;
```

- Méthode alternative pour retourner les mêmes résultats :

```
SELECT * FROM table1, table2;
```

INNER JOIN

- La commande INNER JOIN, aussi appelée EQUIJOIN, est un type de jointures très communes pour lier plusieurs tables entre-elles. Cette commande retourne les enregistrements lorsqu'il y a au moins une ligne dans chaque colonne qui correspond à la condition.

```
SELECT * FROM table1 INNER JOIN table2 ON  
table1.id = table2.fk_id;
```

```
SELECT * FROM table1 INNER JOIN table2  
WHERE table1.id = table2.fk_id;
```

LEFT JOIN

- La commande LEFT JOIN (aussi appelée LEFT OUTER JOIN) est un type de jointure entre 2 tables. Cela permet de lister tous les résultats de la table de gauche même s'il n'y a pas de correspondance dans la deuxième tables.

```
SELECT * FROM table1 LEFT (OUTER) JOIN  
table2 ON table1.id = table2.fk_id;
```

RIGHT JOIN

- La commande RIGHT JOIN (ou RIGHT OUTER JOIN) est un type de jointure entre 2 tables qui permet de retourner tous les enregistrements de la table de droite (right = droite) même s'il n'y a pas de correspondance avec la table de gauche. S'il y a un enregistrement de la table de droite qui ne trouve pas de correspondance dans la table de gauche, alors les colonnes de la table de gauche auront NULL pour valeur.

```
SELECT * FROM table1 RIGHT OUTER JOIN  
table2 ON table1.id = table2.fk_id;
```

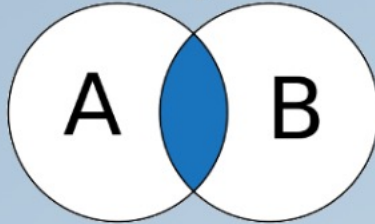
FULL JOIN

- La commande FULL JOIN (ou FULL OUTER JOIN) permet de faire une jointure entre 2 tables. L'utilisation de cette commande permet de combiner les résultats des 2 tables, les associer entre eux grâce à une condition et remplir avec des valeurs NULL si la condition n'est pas respectée.

```
SELECT * FROM table1 FULL OUTER JOIN  
table2 ON table1.id = table2.fk_id;
```

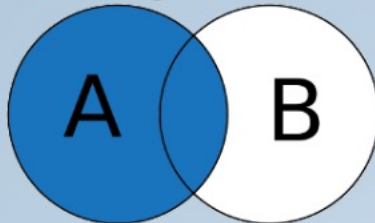

Les jointures

INNER JOIN



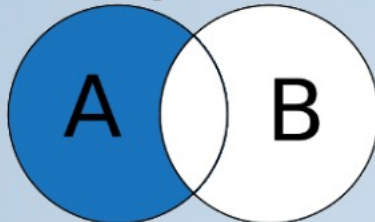
```
SELECT *  
FROM A  
INNER JOIN B ON A.key = B.key
```

LEFT JOIN



```
SELECT *  
FROM A  
LEFT JOIN B ON A.key = B.key
```

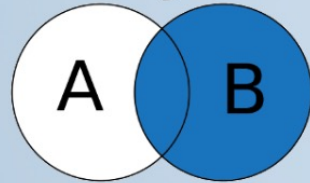
LEFT JOIN (sans l'intersection de B)



```
SELECT *  
FROM A  
LEFT JOIN B ON A.key = B.key  
WHERE B.key IS NULL
```

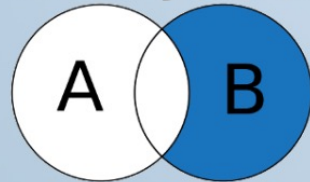
Les jointures

RIGHT JOIN



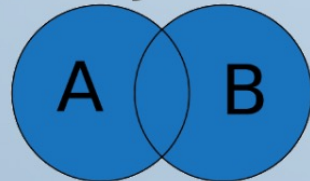
```
SELECT *  
FROM A  
RIGHT JOIN B ON A.key = B.key
```

RIGHT JOIN (sans l'intersection de A)



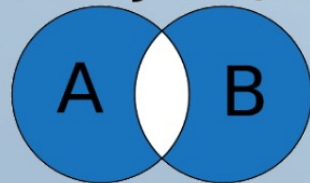
```
SELECT *  
FROM A  
RIGHT JOIN B ON A.key = B.key  
WHERE B.key IS NULL
```

FULL JOIN



```
SELECT *  
FROM A  
FULL JOIN B ON A.key = B.key
```

FULL JOIN (sans intersection)



```
SELECT *  
FROM A  
FULL JOIN B ON A.key = B.key  
WHERE A.key IS NULL  
OR B.key IS NULL
```

Les jointures

- **SELF JOIN** : permet d'effectuer une jointure d'une table avec elle-même comme si c'était une autre table.

Exemple:

```
SELECT u1.u_id, u1.u_nom, u2.u_id, u2.u_nom
FROM utilisateur as u1 LEFT OUTER JOIN utilisateur
as u2 ON u2.u_manager_id = u1.u_id.
```

- **Remarque:** Ici la jointure est effectuée avec un LEFT JOIN, mais il est aussi possible de l'effectuer avec d'autres types de jointures.

```
SELECT column_name(s)
FROM table1 T1, table1 T2
WHERE condition;
```

Les jointures

- **NATURAL JOIN** : jointure naturelle entre 2 tables s'il y a au moins une colonne qui porte le même nom entre les 2 tables SQL.

```
SELECT ...
```

```
FROM <table gauche> NATURAL JOIN <table droite>  
[USING <noms de colonnes>]
```

- Exemple:

```
SELECT CLI_NOM, TEL_NUMERO  
FROM   T_CLIENT  
       NATURAL JOIN T_TELEPHONE  
       USING (CLI_ID)
```

Les jointures

```
SELECT CLI_NOM, TEL_NUMERO  
FROM T_CLIENT C LEFT OUTER JOIN T_TELEPHONE T ON  
C.CLI_ID = T.CLI_ID  
WHERE TYP_CODE = 'FAX' OR TYP_CODE IS NULL
```

Ou encore

```
SELECT CLI_NOM, TEL_NUMERO  
FROM T_CLIENT C LEFT OUTER JOIN T_TELEPHONE T ON  
C.CLI_ID = T.CLI_ID AND TYP_CODE IS NULL
```

Sous-interrogation dans la commande SELECT

```
SELECT ... FROM ...
```

```
WHERE <champ> = (requête SQL : sous-interrogation);
```

- Le résultat de la requête entre parenthèses doit être **de même type** que le champ et **la requête entre parenthèses doit retourner au plus une seule ligne** mais peut comporter plus qu'un champ. Dans ce cas, la sous-requête doit porter sur les mêmes types de champs.
- Exemple:

```
SELECT nom
```

```
FROM Employe
```

```
WHERE travail = (SELECT travail FROM Employe WHERE  
nom = 'Dupont') ;
```

Sous-interrogation dans la commande SELECT

- Pour comparer une valeur supérieure ou égale à toutes (ou à une des) valeurs d'un ensemble donné, les opérateurs ALL et ANY permettent de le faire de manière globale:
 - ALL compare à toutes les valeurs pour que le prédicat soit VRAI,
 - ANY est vrai si, au moins une valeur de l'ensemble répond VRAI à la comparaison.
- Il est également possible de tester la non-appartenance à un ensemble (NOT IN qui correspond au $\neq$ ALL, différent de tous les éléments de l'ensemble).

```
SELECT * FROM EMPLOYE
```

```
WHERE departement IN
```

```
(SELECT dep FROM EMPLOYE
```

```
WHERE nom LIKE 'D%');
```

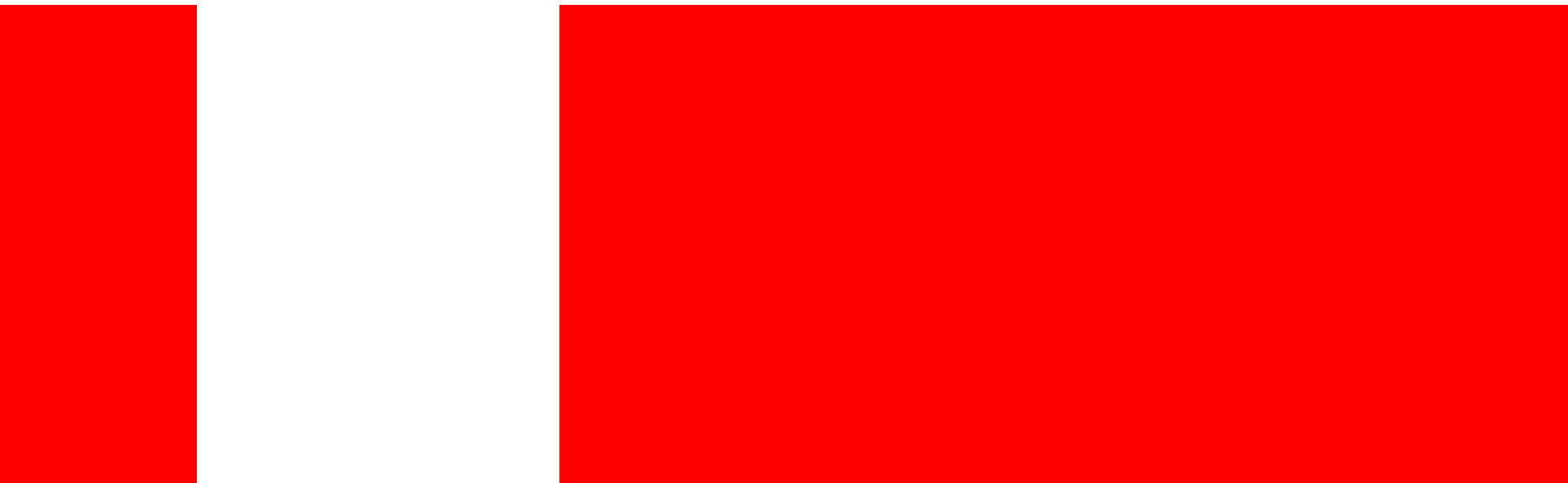
Les opérateurs ensemblistes dans la commande SELECT

```
SELECT ... FROM .... WHERE
```

```
UNION
```

```
SELECT ... FROM .... WHERE ;
```

- La combinaison via un opérateur ensembliste permet un résultat unique des lignes provenant de deux interrogations différentes.



ORACLE®

Questions ???